

CRITERION

LLM Evaluation Platform

Complete Study Guide

Part 1: Concepts A-Z

Part 2: Project Cheat Sheet by Phase

Part 3: 50-Question Quiz Bank

Built with FastAPI · SQLite · Streamlit · MCP · Render

API: <https://criterion-api-c7mf.onrender.com/docs>

UI: <https://criterion-evall.streamlit.app>

PART 1: CONCEPTS A-Z

1. Terminal and Project Navigation

The terminal is where you run all project commands. On a Mac the default shell is zsh. Every command runs relative to your current directory. You must be in the project folder before running `uvicorn`, `pytest`, or `git` commands.

Example:

```
cd ~/Documents/llm-eval-platform
ls
```

Common mistake: Running `uvicorn` from the wrong directory causes `ModuleNotFoundError` because Python cannot find the `api/` package.

2. Virtual Environments

A virtual environment is an isolated Python installation for your project. It keeps your project dependencies separate from your system Python. Always activate it before working.

Example:

```
python3.11 -m venv venv
source venv/bin/activate
# Prompt shows (venv) when active
deactivate # to exit
```

Common mistake: Installing packages without activating `venv` installs them globally. They work locally but Render will not have them because it installs from `requirements.txt` into its own environment.

3. requirements.txt

A plain text file listing every package your project needs. Render reads this file to install dependencies before starting your app. Every `pip` install must be followed by adding the package here.

Example:

```
fastapi
uvicorn
pydantic
python-dotenv
streamlit
requests
pytest
httpx
scikit-learn
mcp
```

Common mistake: Forgetting to add a package after `pip` install. Works locally because the package is in your `venv`. Fails on Render with `ModuleNotFoundError`.

4. .gitignore

A file that tells Git which files and folders to skip. Critical for protecting API keys and skipping large or generated folders.

Example:

```
venv/  
.env  
__pycache__/  
*.pyc  
data/eval.db  
.DS_Store  
**/.DS_Store
```

Common mistake: Pushing .env to GitHub makes your API keys public. Anyone can read them and use your quota or get your account banned.

5. .env and Environment Variables

A .env file stores secret values like API keys locally. python-dotenv reads it and loads the values into the environment. The app reads them with os.getenv().

Example:

```
# .env file  
GEMINI_API_KEY=your_key_here  
  
# In Python  
from dotenv import load_dotenv  
import os  
load_dotenv()  
key = os.getenv('GEMINI_API_KEY')
```

Common mistake: Calling load_dotenv() after other modules have already tried to read environment variables. Those modules get None. Always call load_dotenv() at the very top of main.py.

6. Git Basics

Git tracks changes to your code. GitHub stores those changes online. Every session ends with git add, commit, and push.

Example:

```
git init  
git remote add origin https://github.com/user/repo.git  
git add .  
git commit -m 'feat: add exact_match grader'  
git push origin main  
  
# Tags  
git tag v1.0  
git push origin v1.0
```

Common mistake: git push rejected when remote has commits you do not have locally. Fix: git pull origin main --rebase then push again.

7. FastAPI Basics

FastAPI is a Python library for building REST APIs. You write a function, add a decorator like `@app.get('/health')`, and FastAPI turns it into a live web endpoint. It auto-generates interactive docs at `/docs`.

Example:

```
from fastapi import FastAPI
app = FastAPI()

@app.get('/health')
def health():
    return {'status': 'ok', 'version': '0.1.0'}
```

Common mistake: Running `uvicorn` without `--reload` means you must restart the server manually after every code change.

8. APIRouter

`APIRouter` groups related endpoints in their own file. You create a router in `api/routes/jobs.py` and register it in `main.py` with `app.include_router()`. Without `include_router`, the endpoints are invisible.

Example:

```
# api/routes/jobs.py
from fastapi import APIRouter
router = APIRouter(prefix='/jobs', tags=['jobs'])

@router.post('')
def submit_job(job: EvalJob):
    return run_eval(job)

# api/main.py
from api.routes.jobs import router as jobs_router
app.include_router(jobs_router)
```

Common mistake: Defining router after the route functions. Router must be defined before any `@router` decorator is used.

9. Route Ordering in FastAPI

FastAPI matches routes top to bottom in the order they are registered. Parameterised routes like `/jobs/{job_id}` must always come after specific routes like `/jobs/leaderboard`.

Example:

```
# CORRECT order in jobs.py
@router.get('/leaderboard') # specific first
@router.post('/batch') # specific first
@router.post('/failures/cluster') # specific first
@router.get('/{job_id}') # parameterised last
```

Common mistake: Putting `/job_id` before `/leaderboard`. FastAPI treats the word 'leaderboard' as a `job_id` and the `leaderboard` route never fires. Returns 404 or wrong data.

10. Pydantic Models

Pydantic models define the shape of data the API expects. FastAPI uses them to validate incoming JSON automatically. If a required field is missing, FastAPI returns a 422 error before your code runs.

Example:

```
from pydantic import BaseModel

class EvalJob(BaseModel):
    input: str
    prediction: str
    reference: str
    grader_name: str = 'exact_match'
    model_name: str = 'unknown'
```

Common mistake: Sending `None` from Streamlit for an optional field bypasses the Pydantic default. Always use `model_name` or 'unknown' in the payload, not `model_name` or `None`.

11. Startup Events

A FastAPI startup event runs once when the server first starts, before any requests are handled. Used for setup work like creating database tables.

Example:

```
@app.on_event('startup')
def startup():
    create_tables()
```

Common mistake: Not calling `create_tables()` on startup. If the table does not exist, the first `POST /jobs` call fails with `OperationalError: no such table`.

12. The 4 Graders

A grader is a function that takes a prediction and a reference, compares them, and returns a score. All graders live in `core/graders.py` and are called via a dispatch table in `core/runner.py`.

Example:

```
# exact_match
def exact_match(prediction, reference):
    return 1.0 if prediction.strip().lower() == reference.strip().lower() else 0.0

# contains_match
def contains_match(prediction, reference):
    return 1.0 if reference.lower() in prediction.lower() else 0.0

# regex_match
import re
```

```
def regex_match(prediction, pattern):
    return 1.0 if re.search(pattern, prediction) else 0.0
```

```
# llm_judge: calls Gemini API, returns float 0.0-1.0 + reasoning string
```

Common mistake: `exact_match` fails on whitespace differences. Always `strip()` and `lower()` both strings before comparing.

13. Dispatch Table

A dispatch table is a dictionary that maps string keys to functions. It replaces long if/else chains. When a job comes in, the runner looks up the grader function by name and calls it.

Example:

```
GRADERS = {
    'exact_match': exact_match,
    'contains_match': contains_match,
    'regex_match': regex_match,
    'llm_judge': llm_judge,
}
```

```
# In runner.py
grader_fn = GRADERS[job.grader_name]
result = grader_fn(job.prediction, job.reference)
```

Common mistake: Using if/else instead of a dispatch table. Adding a new grader requires editing the if/else chain in multiple places. With a dispatch table you only add one line.

14. SQLite and Persistence

SQLite is a file-based database. No server needed. The entire database lives in `data/eval.db`. Every eval job is saved there after grading.

Example:

```
import sqlite3

def get_db_connection():
    conn = sqlite3.connect('data/eval.db')
    conn.row_factory = sqlite3.Row
    return conn

# Always close the connection
conn = get_db_connection()
row = conn.execute('SELECT * FROM jobs WHERE id = ?', (job_id,)).fetchone()
conn.close()
```

Common mistake: Forgetting `conn.commit()` after INSERT or UPDATE. Writes are lost when the connection closes.

15. `row_factory = sqlite3.Row`

By default SQLite returns rows as plain tuples. Setting `row_factory = sqlite3.Row` makes rows behave like dictionaries so you can access values by column name instead of index.

Example:

```
conn.row_factory = sqlite3.Row
row = conn.execute('SELECT * FROM jobs WHERE id = ?', (1,)).fetchone()
print(row['score']) # works
print(row[5]) # also works but fragile
return dict(row) # convert to plain dict for JSON response
```

Common mistake: Returning a `sqlite3.Row` object directly from a FastAPI route. FastAPI cannot serialize it to JSON. Always convert with `dict(row)`.

16. SQL Injection Prevention

SQL injection is an attack where malicious input modifies your SQL query. Always use `?` placeholders to pass values into queries. Never use f-strings or string concatenation for query values.

Example:

```
# SAFE
conn.execute('SELECT * FROM jobs WHERE id = ?', (job_id,))

# DANGEROUS - never do this
conn.execute(f'SELECT * FROM jobs WHERE id = {job_id}')
```

Common mistake: Using f-strings in SQL queries. A malicious `job_id` like `'1 OR 1=1'` would return all rows.

17. GROUP BY vs ORDER BY

GROUP BY collapses multiple rows with the same value into one summary row. It is required before aggregate functions like AVG, COUNT, SUM. ORDER BY only sorts the results. It does not collapse or aggregate anything.

Example:

```
-- Leaderboard query
SELECT
model_name,
COUNT(*) as total_runs,
ROUND(AVG(score), 3) as avg_score,
ROUND(SUM(passed) * 100.0 / COUNT(*), 1) as pass_rate
FROM jobs
WHERE model_name IS NOT NULL AND model_name != ''
GROUP BY model_name
ORDER BY avg_score DESC
```

Common mistake: Using ORDER BY without GROUP BY and expecting aggregates. ORDER BY only sorts. You get one row per job, not one row per model.

18. TF-IDF Clustering

TfidfVectorizer converts text into sparse numerical vectors based on word frequency, weighted by how rare each word is across the corpus. KMeans then groups those vectors into clusters. Similar failure patterns land in the same cluster.

Example:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans

def cluster_failures(texts, n_clusters=3):
    if len(texts) < n_clusters:
        n_clusters = len(texts)
    vectorizer = TfidfVectorizer()
    embeddings = vectorizer.fit_transform(texts).toarray()
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    labels = kmeans.fit_predict(embeddings)
    return [{'text': t, 'cluster': int(l)} for t, l in zip(texts, labels)]
```

Common mistake: Setting `n_clusters` higher than `len(texts)`. KMeans crashes. Always cap `n_clusters` to `len(texts)` first.

19. Lazy Loading

Lazy loading means a resource is not loaded until it is actually needed. For heavy models, this prevents them from consuming memory at startup.

Example:

```
model = None

def get_model():
    global model
    if model is None:
        model = load_heavy_model()
    return model

def cluster_failures(texts):
    embeddings = get_model().encode(texts)
```

Common mistake: Loading the model at module level (`model = SentenceTransformer(...)`). This runs at import time and consumes memory immediately when FastAPI starts, causing Render's 512MB free tier to crash.

20. MCP and stdio Mode

MCP (Model Context Protocol) is a standard that lets AI agents discover and call your API as a tool. FastMCP wraps Python functions with `@mcp.tool()` and exposes them. The server communicates over stdin/stdout (stdio mode).

Example:

```
from mcp.server.fastmcp import FastMCP
import httpx
import os

API_URL = os.getenv('API_URL', 'http://localhost:8000')
```



```

mcp = FastMCP('Criterion')

@mcp.tool()
def run_eval(input: str, prediction: str, reference: str,
grader_name: str = 'exact_match') -> dict:
    '''Submit an eval job and return the result.'''
    payload = {'input': input, 'prediction': prediction,
'reference': reference, 'grader_name': grader_name}
    return httpx.post(f'{API_URL}/jobs', json=payload).json()

if __name__ == '__main__':
    mcp.run()

```

Common mistake: Thinking the server crashed because the prompt returns immediately after `python mcp/server.py`. This is correct. `stdio` mode waits for input from a caller. It exits when nothing is calling it.

21. 0.0.0.0 vs 127.0.0.1

127.0.0.1 (localhost) only accepts connections from the same machine. 0.0.0.0 accepts connections from any network interface. Render routes external traffic from outside your container, so the app must listen on 0.0.0.0.

Example:

```

# Local development
uvicorn api.main:app --reload
# (defaults to 127.0.0.1:8000)

# Render production start command
uvicorn api.main:app --host 0.0.0.0 --port 10000

```

Common mistake: Using `--host 127.0.0.1` on Render. Render's traffic never reaches your container. The service shows as deployed but every request times out. Render reports: No open ports detected.

22. Render Deployment

Render is a cloud platform that hosts web services. It reads your GitHub repo, installs `requirements.txt`, and runs your start command. The free tier has 512MB RAM and spins down after 15 minutes of inactivity.

Example:

```

# Render settings
Build Command: pip install -r requirements.txt
Start Command: uvicorn api.main:app --host 0.0.0.0 --port 10000

# Environment variables in Render dashboard
GEMINI_API_KEY = your_key_here

```

Common mistake: Putting secrets in `.env` and pushing to GitHub instead of adding them in Render's Environment tab. The `.env` file is in `.gitignore` so Render never sees it.

23. Streamlit Cloud Deployment

Streamlit Cloud deploys ui/app.py directly from your GitHub repo. Secrets like API_URL are added in the app's Settings under Secrets, not in the code.

Example:

```
# In Streamlit Cloud Secrets
API_URL = 'https://criterion-api-c7mf.onrender.com'

# In ui/app.py
import os
API_URL = os.getenv('API_URL', 'http://localhost:8000')
```

Common mistake: Hardcoding the localhost URL in ui/app.py. Works locally but every API call from Streamlit Cloud fails with `httpx.ConnectError` because localhost:8000 does not exist in the cloud.

24. pytest

pytest is the standard Python testing framework. Test files go in tests/. Every function starting with test_ is discovered and run automatically.

Example:

```
# tests/test_graders.py
from core.graders import exact_match, contains_match

def test_exact_match_pass():
    assert exact_match('Paris', 'Paris') == 1.0

def test_exact_match_fail():
    assert exact_match('paris', 'Paris') == 1.0 # strips and lowercases

# Run
pytest tests/ -v
```

Common mistake: Running pytest from the wrong directory. Python cannot find core/ and raises `ModuleNotFoundError`. Fix: add `pytest.ini` with `pythonpath = .` in the project root.

25. Homebrew and Python Version Management

Homebrew is the standard package manager for macOS. Use it to install system tools like Python 3.11. macOS ships with Python 3.9 which is too old for the mcp package.

Example:

```
# Install Homebrew
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Add to PATH
eval "$(/opt/homebrew/bin/brew shellenv)"

# Install Python 3.11
brew install python@3.11
```

```
# Create venv with Python 3.11  
/opt/homebrew/bin/python3.11 -m venv venv
```

Common mistake: Running `python3 -m venv venv` on a Mac without Homebrew. Gets Python 3.9.6 which cannot install mcp. Always specify the full path to the Homebrew Python.

PART 2: PROJECT CHEAT SHEET BY PHASE

Phase: Setup

What was built	Folder structure, virtual environment, GitHub repo, .env, .gitignore, FastAPI skeleton with GET /health
Key files	requirements.txt api/main.py api/__init__.py core/__init__.py api/routes/__init__.py .env .gitignore
Key commands	<pre>mkdir llm-eval-platform && cd llm-eval-platform /opt/homebrew/bin/python3.11 -m venv venv source venv/bin/activate pip install fastapi uvicorn pydantic python-dotenv pip install -r requirements.txt uvicorn api.main:app --reload git init && git remote add origin <url> git add . && git commit -m 'feat: project setup' git push origin main</pre>
What broke and fix	Attribute 'app' not found. Caused by running uvicorn before saving code to the file. Fix: always save the file first.

Phase: Graders

What was built	4 graders in core/graders.py, POST /jobs route, dispatch table in runner.py, 7 pytest tests
Key files	core/graders.py core/runner.py core/models.py api/routes/jobs.py api/models.py tests/test_graders.py pytest.ini
Key commands	<pre>pip install pytest httpx pytest tests/ -v curl -X POST http://localhost:8000/jobs -H 'Content-Type: application/json' -d '{"input":</pre>
What broke and fix	404 model not found for gemini-1.5-flash. Model name changed on the API. Fix: called ListModels endpoint and switched

Phase: Persistence

What was built	SQLite database at data/eval.db, create_tables(), save_job_result(), load_job_history(), GET /history
Key files	core/runner.py api/routes/history.py data/eval.db

Key commands	# Tables created automatically on startup curl http://localhost:8000/history
What broke and fix	OperationalError: no such table. create_tables() was not wired to startup event. Fix: add @app.on_event('startup') calling

Phase: Leaderboard

What was built	GET /jobs/leaderboard with GROUP BY SQL, POST /jobs/batch, GET /jobs/{job_id} with 404 handling
Key files	api/routes/jobs.py core/runner.py
Key commands	curl http://localhost:8000/jobs/leaderboard curl -X POST http://localhost:8000/jobs/batch -H 'Content-Type: application/json' -d '{
What broke and fix	Leaderboard returned 404. /jobs/leaderboard was registered after /jobs/{job_id}. FastAPI matched 'leaderboard' as a job.

Phase: Clustering

What was built	POST /jobs/failures/cluster using TF-IDF and KMeans
Key files	core/clustering.py api/routes/jobs.py
Key commands	pip install scikit-learn curl -X POST http://localhost:8000/jobs/failures/cluster -H 'Content-Type: application/j
What broke and fix	Originally used sentence-transformers which pulled in torch (~500MB). Crashed Render's 512MB free tier even with lazy

Phase: UI

What was built	Streamlit 3-page UI with F1 design: Submit Evaluation, History, Leaderboard. All pages call the FastAPI API.
Key files	ui/app.py
Key commands	pip install streamlit httpx streamlit run ui/app.py # Open http://localhost:8501
What broke and fix	httpx.ConnectError on every page. Streamlit was running but FastAPI was not. Fix: always start uvicorn in Tab 1 before s

Phase: MCP

What was built	mcp/server.py with run_eval and get_leaderboard tools using FastMCP
Key files	mcp/server.py requirements.txt
Key commands	pip install mcp httpx python mcp/server.py # Prompt returns immediately - this is correct (stdio mode)
What broke and fix	mcp package requires Python 3.10+. macOS system Python is 3.9.6. Fix: install Homebrew, brew install python@3.11, r

Phase: Deploy

What was built	FastAPI on Render, Streamlit on Streamlit Cloud, v1.0 tagged on GitHub
Key files	requirements.txt ui/app.py core/clustering.py
Key commands	# Render start command uvicorn api.main:app --host 0.0.0.0 --port 10000 # Tag git tag v1.0 && git push origin v1.0 # Verify curl https://criterion-api-c7mf.onrender.com/health
What broke and fix	Render OOM (512MB) from sentence-transformers/torch. Fix: replaced with TF-IDF. Also: Streamlit ConnectError because

Phase: Launch

What was built	Complete README.md with live URLs, stack, endpoints, architecture, run instructions. LinkedIn post scheduled.
Key files	README.md
Key commands	git add README.md git commit -m 'docs: complete README' git push origin main
What broke and fix	model_name sent as None from Streamlit when field left empty. Bypassed Pydantic default and stored NULL in DB. Filter

PART 3: QUIZ BANK — 50 QUESTIONS

Answer all questions before checking the answer key at the end.

1. [Setup] What command activates the virtual environment?

- A. python venv activate
- B. source venv/bin/activate
- C. activate venv
- D. venv start

2. [Setup] What does .gitignore do?

Answer: _____

3. [Setup] Why must .env never go to GitHub?

Answer: _____

4. [Setup] What does pip install -r requirements.txt do?

Answer: _____

5. [Setup] What is the correct FastAPI startup command locally?

- A. python api/main.py
- B. fastapi run
- C. uvicorn api.main:app --reload
- D. flask run

6. [Git] What does git add . do?

Answer: _____

7. [Git] Push was rejected with 'remote contains work you do not have'. What do you run?

- A. git push --force
- B. git pull origin main --rebase then push
- C. git reset --hard
- D. git clone again

8. [Git] How do you create and push a tag?

Answer: _____

9. [FastAPI] What HTTP status code means the request succeeded?

- A. 404
- B. 422
- C. 200
- D. 500

10. [FastAPI] What does a 422 response mean?

Answer: _____

11. [FastAPI] Where does FastAPI auto-generate interactive API docs?

- A. /api
- B. /swagger
- C. /docs
- D. /openapi

12. [FastAPI] Why must `app.include_router()` be called in `main.py`?

Answer: _____

13. [FastAPI] What decorator runs a function once when the server starts?

- A. `@app.startup`
- B. `@app.on_event('startup')`
- C. `@app.init`
- D. `@app.before_request`

14. [Routes] You have `GET /jobs/leaderboard` and `GET /jobs/{job_id}`. Which must be registered first?

- A. `/job_id` first
- B. `/leaderboard` first
- C. Order does not matter
- D. They cannot coexist

15. [Routes] What happens if `/job_id` is registered before `/leaderboard`?

Answer: _____

16. [Pydantic] EvalJob has model_name: str = 'unknown'. What does the default do?

Answer: _____

17. [Pydantic] Streamlit sends model_name: None. Does the Pydantic default kick in?

A. Yes

B. No, None bypasses the default

18. [Graders] Which grader calls an external API?

A. exact_match

B. contains_match

C. regex_match

D. llm_judge

19. [Graders] What does exact_match return for ('Paris ', 'paris')?

A. 0.0

B. 1.0

C. Error

D. None

20. [Graders] What is a dispatch table?

Answer: _____

21. [Graders] contains_match('The capital is Paris', 'Paris') returns:

A. 0.0

B. 1.0

C. 0.5

D. Error

22. [Graders] llm_judge returns a score and what else?

Answer: _____

23. [SQLite] What does conn.row_factory = sqlite3.Row do?

Answer: _____

24. [SQLite] Why must you call conn.commit() after an INSERT?

Answer: _____

25. [SQLite] What is SQL injection and how do you prevent it?

Answer: _____

26. [SQLite] What does CREATE TABLE IF NOT EXISTS do?

Answer: _____

27. [SQLite] Why use dict(row) before returning from a FastAPI route?

Answer: _____

28. [SQL] What does GROUP BY model_name do?

Answer: _____

29. [SQL] What is the difference between GROUP BY and ORDER BY?

Answer: _____

30. [SQL] SUM(passed) * 100.0 / COUNT(*) gives you what?

Answer: _____

31. [Clustering] What does TfidfVectorizer do?

Answer: _____

32. [Clustering] Why must n_clusters be capped at len(texts)?

Answer: _____

33. [Clustering] What does random_state=42 do in KMeans?

Answer: _____

34. [Clustering] Why was sentence-transformers replaced with TF-IDF?

Answer: _____

35. [MCP] What does @mcp.tool() do?

Answer: _____

36. [MCP] Why does mcp/server.py use httpx instead of importing core/ directly?

Answer: _____

37. [MCP] python mcp/server.py runs and the prompt returns immediately. Is this an error?

A. Yes, it crashed

B. No, stdio mode exits when nothing is calling it

38. [MCP] Name the two tools exposed by mcp/server.py.

Answer: _____

39. [Deploy] What is the Render start command for FastAPI?

Answer: _____

40. [Deploy] Why --host 0.0.0.0 and not 127.0.0.1 on Render?

Answer: _____

41. [Deploy] Render shows 'No open ports detected'. What is wrong?

A. Wrong port number

B. App is using 127.0.0.1 instead of 0.0.0.0

C. Missing requirements.txt

D. Both A and B

42. [Deploy] Where do you add GEMINI_API_KEY on Render?

A. In .env pushed to GitHub

B. In requirements.txt

C. In Render's Environment tab

D. In main.py

43. [Deploy] Streamlit Cloud shows httpx.ConnectError. What is wrong?

Answer: _____

44. [Deploy] What does the free Render tier do after 15 minutes of inactivity?

Answer: _____

45. [Architecture] What is the architecture rule for Criterion?

Answer: _____

46. [Architecture] Why does Streamlit use httpx instead of importing core/ directly?

Answer: _____

47. [Architecture] GET /history vs GET /jobs/{job_id} — when does an agent use each?

Answer: _____

48. [Misc] What is the production API URL for Criterion?

Answer: _____

49. [Misc] What is the production UI URL for Criterion?

Answer: _____

50. [Misc] What does v1.0 represent in this project?

Answer: _____

ANSWER KEY

Q1: B

Q2: Tells Git which files and folders to skip. Prevents venv/ and .env from being pushed to GitHub.

Q3: It contains API keys. Pushing it makes them public. Anyone can read them and use your quota or get your account banned.

Q4: Installs every package listed in requirements.txt in one command. Render runs this during the build step.

Q5: C

Q6: Stages all changed files. Tells Git these are the files to include in the next commit.

Q7: B

Q8: git tag v1.0 then git push origin v1.0. Tags do not push automatically with git push origin main.

Q9: C

Q10: Unprocessable Entity. The request body does not match the Pydantic model. A required field is missing or the wrong type.

Q11: C

Q12: Without it, the routes defined in the router file are invisible. FastAPI only sees routes that are registered.

Q13: B

Q14: B

Q15: FastAPI matches 'leaderboard' as a job_id. The leaderboard route never fires. Returns 404 or wrong data.

Q16: If model_name is not included in the request body, it defaults to 'unknown'. Prevents NULL in the database.

Q17: B

Q18: D

Q19: B

Q20: A dictionary mapping grader name strings to grader functions. Replaces if/else chains. Adding a new grader only requires one line.

Q21: B

Q22: A reasoning string explaining why the score was given.

Q23: Makes rows behave like dictionaries so you can access values by column name like row['score'] instead of index.

Q24: Without commit(), writes are not saved to disk. They are lost when the connection closes.

Q25: An attack where malicious input modifies your SQL query. Prevent with ? placeholders, never f-strings.

Q26: Creates the table only if it does not already exist. Safe to call every time the server starts without wiping existing data.

Q27: FastAPI cannot serialize `sqlite3.Row` to JSON. `dict(row)` converts it to a plain Python dictionary.

Q28: Collapses all rows with the same `model_name` into one summary row. Required before AVG, COUNT, SUM.

Q29: GROUP BY collapses rows and enables aggregates. ORDER BY only sorts results. ORDER BY does not aggregate.

Q30: The pass rate as a percentage. `passed` is 1 for pass and 0 for fail, so summing it gives total passes.

Q31: Converts text into sparse numerical vectors based on word frequency, weighted by how rare each word is across the corpus.

Q32: KMeans crashes if you request more clusters than data points. Cap `n_clusters = min(n_clusters, len(texts))`.

Q33: Makes clustering reproducible. Same input always produces the same cluster assignments.

Q34: `sentence-transformers` pulls in torch (~500MB). Render free tier has 512MB RAM total. TF-IDF uses kilobytes and is already in scikit-learn.

Q35: Decorates a Python function and exposes it as a callable tool for MCP-compatible agents like Claude.

Q36: The MCP server is a separate process. It cannot share memory with FastAPI. It must call the API over HTTP.

Q37: B

Q38: `run_eval` (submits an eval job) and `get_leaderboard` (returns ranked model scores).

Q39: `uvicorn api.main:app --host 0.0.0.0 --port 10000`

Q40: 127.0.0.1 only accepts local connections. 0.0.0.0 accepts external traffic. Render routes traffic from outside the container.

Q41: D

Q42: C

Q43: `API_URL` is pointing to `localhost:8000` which does not exist in the cloud. Fix: add `API_URL` secret in Streamlit Cloud settings pointing to the Render URL.

Q44: Spins down the container. The first request after that wakes it up, causing 50+ seconds of delay.

Q45: The API is the product. Streamlit calls the API. Business logic never lives in the UI layer.

Q46: Streamlit is a display layer only. It must call the API over HTTP like any other client. This keeps the API as the single source of truth.

Q47: GET `/jobs/{job_id}` when you know the exact ID and want one result. GET `/history` when you want a list of recent runs without a specific ID.

Q48: <https://criterion-api-c7mf.onrender.com>

Q49: <https://criterion-evall.streamlit.app>

Q50: The first production-ready version. API live on Render, UI live on Streamlit Cloud, all endpoints working, README complete.

Criterion — Built by Shihab Uddin Alvi

API-first LLM Evaluation Platform